DEPT

Backend Node Assignment

Sebastian Mihai (mihaisebastian01@gmail.com)

Tech Stack

Requirements

All Tips + Bonuses + Mandatory requirements are met. *Notes*:

1) The website is a SPA, and also implement such that it is fast. Backend written in TypeScript (no time for in-depth validation and TS types matching)

2) Since my time was quite constrained, the project is not 100% secure, so please do not rely on the data being stored completely safely. I added JWT and CSRF tokens + XSS validation, but I am not sure they are working properly.

3) Since I had some problems with Google OAuth2 after deployment and I had to delete some middlewares (csrf, jwt - wise), I will also make some small tweaks (+ project documentation) after I send this email, but it won't be extensive work, so I consider the project to be completed at the moment that I'm sending this email.

4) Please do not exploit the requests that are being sent from the 'Browse' page. IMDb required a specific request in order to grant me access to their API, and the response was going to come after 48h of submission, and as I had limited time I instead used RapidAPI, which charges money/request sent. My approximation is that the page can be reloaded ~20 times/day.

5) The project manual (including documentation) will be added as a PDF to both repositories.If anything goes wrong while trying to access the project online, please let me know.

6) I did not use memoization because you cannot memorize values that are roughly similar to each other, and those values' states are

not dependent on previous states. Example: if you search for ‘Blacklist 2013’ based on ‘Blacklist 2012’, the result will not be the same. Therefore, we will take a broader caching technique, namely server-side HTTP caching with the memory cache package

7) Videos are shown 5 in total, split in 2 pages. Rendered 3 per page using pagination (styling from bootstrap). Results are cached on the backend as a middleware validator. This can be noticed after reloading a page for the second time.

8) Instead, I tried to use a third party package. If I had to do it by myself, I would have used the Dynamic Programming approach for word wrapping.

9) Frontend: Router, ContextAPI, local Token + validation from the Google OAuth2 for authentication, SendGrid for the Contact page, loading modals based on states, Bootstrap for styling.

Backend

1) From top-to-bottom, given the directories: Auth has authentication middlewares, CSRF for matching with the website url and making sure the protocol is HTTPS. JWT – basic behavior. Validation checks for tokens and XSS does sanitization.

2) Controller: Has APIs for connection with the Frontend and MongoDB Atlas. The Mongoose ODM is used for communication. As of now, the collection, database & documents are saved but not stored on the database because I am not 100%

sure the overall app is 100% secure. This can be tested with requests however.

3) Search API has all functions executed in the Express Router.

4) Misc: Intended to use EJS for error displaying. Mail-sender has the SendGrid communication protocol inside which is used for sending mails.

5) Models: Has the MongoDB User model.

6) Routes: Express router, containing auth and validation middlewares, as well as all possible routes to touch our backend.

7) Utils: Caching – done with memory-cache for the reasons explained in the previous paragraph. Also TTL.

8) App.ts has CORS headers set, methods and IP whitelist.

9) Code is not clean since this was not my intended prototype, and time as quite short as I intended to program the entire project, not just the shorter version of it as it was mentioned.

I had a lot of fun writing this project and I pushed myself to finish the project by the intended deadline, and the last push was done on Friday (unless there is a deployment issue found in the meantime)

Best regards,
Sebastian